Circles Token Offer Audit

Gnosis Ltd - Report by Côme du Crest

2025-09-21

Table of contents

- Table of contents
- Circles Token Offer Audit
  - Scope
  - Context
  - Status
- Issues
  - [Fixed][Low] Incorrect time comparison for offer start and end
  - [Fixed][Info] Offers trusting tokens on hub is not strictly necessary
  - [Fixed][Info] Two cycle offers open at the same time

Circles Token Offer Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis Ltd.

Scope

The scope includes all contracts within `aboutcircles/circles-token-offer` as of commit `0x36ecdb3`.

The code has been reviewed as of commit `0x361570b`.

Context

`ERC20TokenOffer` allows to sell ERC20 tokens for CRC tokens of specified token IDs. The amount of tokens that can be bought by the user depends on a weight that is set in `AccountWeightProviderUnbounded` for that user.

`ERC20TokenOfferCycle` can create offers with consecutive offer Ids that are contiguous. The created offers from `OFFERS_START + (OFFER_DURATION * currentId)` and last for `OFFER_DURATION`.

The admin of the offers created by `ERC20TokenOfferCycle` is itself.

Claiming a regular offer is done processed via `ERC20TokenOffer.onERC1155{Batch}Received()` which checks that `from` is within the claim offer limit associated by its weight, and checks that the sent CRC tokens have accepted Ids. It then transfers the ERC20 tokens to the `from` address and the `CRC` tokens to the offer owner.

For cyclic offers, the claiming is started with `ERC20TokenOfferCycle.onERC1155{Batch}Received()` which re-encodes the `from` address in the ERC1155 transfer data to be decoded by `ERC20TokenOffer` before transferring the received CRC tokens to the currently active offer. The `ERC20TokenOffer` will decode the original `from` address and ensure the hub provided `from` address is its owner (`ERC20TokenOfferCycle`) to complete the offer. `ERC20TokenOffer` will send the received CRC tokens back to `ERC20TokenOfferCycle` which will trigger `ERC20TokenOfferCycle.onERC1155{Batch}Received()` once again with `from == currentOfferAddress`. This time, `ERC20TokenOfferCycle` will simply perform accounting and relay the tokens back to its admin.

For cyclic offers, the create `ERC20TokenOffer` can only be interacted with through the `ERC20TokenOfferCycle` that created the offer. This is a good safety measure that prevents most angle of attacks on `ERC20TokenOffer`.

Status

The report has been sent to the core developer. No important security issues have been found, some advice on the code should still be considered.

The code has been updated as of commit `0x361570b` to take into account the provided comments.

Issues

[Fixed][Low] Incorrect time comparison for offer start and end

Summary

The function `depositOfferTokens()` attempts to prevent deposit of tokens after its start by using comparing `block.timestamp` with `OFFER_START` but allows depositing on the exact time `OFFER_START` while offer should be considered started.

The function `withdrawUnclaimedOfferTokens()` prevents withdrawing unclaimed tokens when the offer is not ended yet but it allows withdrawing tokens at the timestamp `OFFER_END` while offer should not be considered ended yet.

Vulnerability Detail

The documentation state that offer activity should be inclusive of start and end time:

```
    /// @notice Offer start timestamp (inclusive).
    uint256 public immutable OFFER_START;

    /// @notice Offer end timestamp (inclusive).
    uint256 public immutable OFFER_END;

    ...

    /// @notice Ensures the offer is currently active by time.
    /// @dev Active if `OFFER_START <= block.timestamp <= OFFER_END`. Reverts
        with {OfferNotActive} otherwise.
    modifier onlyWhileOfferActive() {
        if (block.timestamp < OFFER_START || block.timestamp > OFFER_END) {
            revert OfferNotActive();
        }
        _;
    }
```

However, `depositOfferTokens()` does not revert at `OFFER_START` and `withdrawUnclaimedOfferTokens()` does not revert at `OFFER_END`:

```
    function depositOfferTokens() external onlyOwner {
        if (isOfferTokensDeposited || block.timestamp > OFFER_START) revert
            OfferDepositClosed();
        ...
    }

    ...
    function withdrawUnclaimedOfferTokens() external onlyOwner returns (uint256
         balance) {
```

```
        if (OFFER_END > block.timestamp) revert OfferActive();
        ...
    }
```

Impact

Incoherent handling of time limits in offers.

Code Snippets

https://github.com/aboutcircles/circles-token-offer/blob/36ecdb3efb107b5673b3a711621e30f845d6fae1/src/ERC20TokenOffer.sol#L270

https://github.com/aboutcircles/circles-token-offer/blob/36ecdb3efb107b5673b3a711621e30f845d6fae1/src/ERC20TokenOffer.sol#L289

https://github.com/aboutcircles/circles-token-offer/blob/36ecdb3efb107b5673b3a711621e30f845d6fae1/src/ERC20TokenOffer.sol#L220

Recommendation

Consider offer available in the time frame `[offerStart; offerEnd[` inclusive of `offerStart` and exclusive of `offerEnd` and apply that to all the time comparisons.

Response

The issue has bene fixed in commit `0x361570b` by updating the time comparisons to make offers inclusive of start time and exclusive of end time.

[Fixed][Info] Offers trusting tokens on hub is not strictly necessary

Summary

When created, `ERC20TokenOffer` calls `HUB.trust()` on the array of tokens that can are accepted by this offer. Later, in `onERC1155{batch}Received()` the offer checks that the received tokens are trusted via `HUB.isTrusted()`. A local storage mapping could be used in place of using trust on the hub.

The advantage of using the hub is that the tokens can be received via `Hub.operateFlowMatrix()` which may be desired.

Moreover, when created via `ERC20TokenOfferCycle`, the `ERC20TokenOffer` similarly trust the accepted tokens but will not accept them if sent via `Hub.operateFlowMatrix()` due to `onERC1155{Batch}Received()` not being called from `ERC20TokenOfferCycle`.

Vulnerability Detail

`ERC20TokenOffer` calls `HUB.trust()` for each accepted tokens when created:

```
    constructor(
        ...
        address[] memory acceptedCRC
    ) {
        ...
        for (uint256 i; i < acceptedCRC.length;) {
            HUB.trust(acceptedCRC[i], type(uint96).max);
            unchecked {
                ++i;
            }
        }
    }
```

It verifies that it trusts the received tokens in `onERC1155{Batch}Received()`:

```
    function onERC1155Received(address, address from, uint256 id, uint256 value
        , bytes memory data)
        external
        onlyHub
        onlyWhenOfferTokensDeposited
        onlyWhileOfferActive
        returns (bytes4)
    {
        if (!HUB.isTrusted(address(this), address(uint160(id)))) revert
            InvalidTokenId(id);
        if (CREATED_BY_CYCLE) from = _resolveSender(from, data);
        ...
    }
```

But will revert when tokens are not sent from `ERC20TokenOfferCycle` (owner) when created by cycle:

```
    function _resolveSender(address from, bytes memory data) internal view
        returns (address sender) {
        if (from != OWNER) revert OnlyFromCycle();
        sender = abi.decode(data, (address));
    }
```

Impact

The trust from an offer to a token id does not necessarily mean that the offer can receive the token via `Hub.operateFlowMatrix()` depending on wether it is a cyclic offer or not.

The trust graph of circles in general might get confusing due to cyclic offers.

Code Snippets

https://github.com/aboutcircles/circles-token-offer/blob/36ecdb3efb107b5673b3a711621e30f845d6fae1/src/ERC20TokenOffer.sol#L206-L211

https://github.com/aboutcircles/circles-token-offer/blob/36ecdb3efb107b5673b3a711621e30f845d6fae1/src/ERC20TokenOffer.sol#L366

Recommendation

Acknowledge that this is the desired behaviour or rely on a local storage of accepted tokens in `ERC20TokenOffer` either completely or only for cyclic offers.

Response

Limited the trust from `ERC20TokenOffer` to accepted tokens to the offer end time in commit `0x361570b`.

[Fixed][Info] Two cycle offers open at the same time

Summary

Offers availability are inclusive of start and end time. For cycle offers there are technically two offers available at the exact timestamp `OFFERS_START + (OFFER_DURATION * currentId)`. I.e. both offer `offers[currentId]` and `offers[currentId - 1]` are available when called through `ERC20TokenOffer.isOfferAvailable()`.

I don't think this is an important issue since the offers can only be taken through `ERC20TokenOfferCycle` and not directly due to `ERC20TokenOffer.onERC1155Received()` calling `_resolveSender()` which enforces that `from == OWNER` (the owner being `ERC20TokenOfferCycle`).

Vulnerability Detail

`ERC20TokenOfferCycle` creates offer with contiguous order starts offset each time by `OFFER_DURATION`:

```
    function createNextOffer(uint256 tokenPriceInCRC, uint256 offerLimitInCRC,
        address[] memory _acceptedCRC)
        external
        onlyAdmin
        returns (address nextOffer)
    {
        uint256 currentId = currentOfferId();
        ...
        uint256 offerStart = OFFERS_START + (OFFER_DURATION * currentId);
        ...

        nextOffer = OFFER_FACTORY.createERC20TokenOffer(
            address(ACCOUNT_WEIGHT_PROVIDER),
            address(this),
            OFFER_TOKEN,
            tokenPriceInCRC,
            offerLimitInCRC,
            offerStart, // contiguous start
            OFFER_DURATION,
            offerName,
            _acceptedCRC
        );
        ...
    }
```

`ERC20TokenOffer` are inclusive of start and end time:

```
    constructor(
        ...
        uint256 offerStart,
        uint256 offerDuration,
```

```
        ...
    ) {
        ...
        OFFER_START = offerStart;
        OFFER_END = offerStart + offerDuration;
        ...
    }
    ...

    function isOfferAvailable() external view returns (bool) {
        return OFFER_START <= block.timestamp && OFFER_END >= block.timestamp
            && isOfferTokensDeposited;
    }
```

Impact

Two cyclic offers are available on the exact timestamp where one ends and the other begins. This may cause confusion for third parties integrating on top of the cyclic offer contracts.

Code Snippets

https://github.com/aboutcircles/circles-token-offer/blob/36ecdb3efb107b5673b3a711621e30f845d6fae1/src/ERC20TokenOffer.sol#L182-L221

https://github.com/aboutcircles/circles-token-offer/blob/36ecdb3efb107b5673b3a711621e30f845d6fae1/src/ERC20TokenOfferCycle.sol#L237

Recommendation

Update `ERC20TokenOffer` so that their availabilities are inclusive of start time and not end time.

Response

The issue has bene fixed in commit `0x361570b` by making offers inclusive of start time and exclusive of end time.